

15-418 Assignment 3: Parallel VLSI Wire Routing Report

Jason (jasont2) Catherine (catheri5)

October 8, 2026

1 Shared Baseline and Optimizations

1.1 Exhaustive route generation and cost calculation

We generate routes from an index instead of storing every possible route in a vector. The index tells `route_from_index()` where to place the bends. We construct one route, score it, and discard it unless it is the best so far. This reduces allocation and memory traffic while still checking every legal route.

1.2 Shared locality and construction optimizations

Both modes use the following constant-factor optimizations:

- **Indexed route generation.** We generate one route from its index, score it, and then move to the next index. This avoids allocating and storing every candidate route at once.
- **Horizontal and vertical scoring loops.** We use separate loops for horizontal and vertical segments instead of checking a segment's direction for every cell. This is a small optimization that saves on some checks in the inner most loop.
- **Transposed and Normal Occupancy.** `occupancy[y][x]` makes and `occupancy_columns[x][y]` make horizontal and vertical pieces contiguous in memory. This reduces cache misses since you can load entire lines in the direction that you are traversing.

2 Within-Wires: Design and Performance Debugging Journey

2.1 Implemented optimizations and final parallelization

W mode keeps annealing iterations and the outer wire loop sequential. For one wire, it removes the old route, evaluates legal candidates in parallel, reduces the workers' best routes, and commits one winning route. The implemented optimizations are:

- **Potential wire position per thread** One complete candidate route is the unit of parallel work; candidates for the same removed wire are independent.
- **No candidate vector.** At first, the code generated every route into a vector and parallelized only scoring. The final code constructs, scores, and discards one candidate at a time, eliminating large temporary storage.
- **Indexed direct construction.** A route index determines its bend pattern. Workers therefore construct and score their own candidates, rather than waiting for one thread to build all routes.
- **Exact route counts and random-route construction.** `count_routes()` determines loop bounds and the fallback decision without materializing routes; the random branch constructs only its selected route.
- **Private bests and reusable buffers.** Each worker tracks a private best cost and route, then writes one result into storage allocated once outside the wire loop. This means we only have to synchronize at the end.
- **Thresholded static scheduling.** Searches with fewer than four candidates per requested thread run sequentially. Larger searches use `schedule(static,1)`, assigning complete candidate routes cyclically.
- **Direction-specific scoring and two occupancy views.** The shared horizontal/vertical loops and row-major/transposed occupancy arrays make scans contiguous in the direction being traversed.

2.2 Synchronization and correctness

During one candidate search, workers only read occupancy; the current wire was removed before the search, so each candidate cell uses $(occupancy[y][x] + 1)^2$. Workers write only their own private best route and cost. The work-sharing loop has one barrier, followed by one reduction. Removing and inserting occupancy is sequential between wires. This avoids per-candidate locks, atomics, and critical sections.

2.3 Optimization Decision Making and Tradeoffs

We tested scheduling and granularity on medium input at eight threads. Static scheduling measured 1.5045 s versus 1.5355 s for dynamic scheduling. With static scheduling, chunk one measured 1.4862 s versus 1.5134 s for chunk four; the differences were small, but favored inexpensive candidate-level static scheduling. We therefore retained static, chunk-one scheduling and the small-search fallback rather than adding a dynamic task queue.

The verified final GHC results in Table 7 and Figure 2 show W eight-thread computation speedups of $7.70\times$, $7.20\times$, and $5.95\times$ on few, medium, and abundant inputs. The shared indexed construction and dual occupancy views reduce allocation traffic and improve horizontal and vertical scan locality.

2.4 Limitations against Perfect Speedup

W exposes candidates for only one wire at a time. Some wires have too few candidates to keep all 128 workers busy, and every wire requires a barrier, reduction, and sequential occupancy update before the next search starts, causing large synchronization overhead if the space of possible wires is small compared to amount of wires. Data movement is a contributing cost, but it is not by itself a complete explanation for the W -mode limit. On GHC, total W cache misses do rise as threads are added: the medium case rises from 27.48 million at one thread to 33.35 million at eight, and the abundant case from 114.05 million to 135.90 million. This is evidence of additional cache and memory pressure. However, even with ideal cache behavior, W would still expose only one wire’s candidates at a time and would still pay one barrier, reduction, and serial occupancy update per wire. Those structural costs set a concurrency limit independently of data movement; data movement amplifies it rather than replacing it as an explanation.

We also considered splitting one candidate route’s cell score across several workers, or parallelizing the final reduction of per-worker best routes.

We rejected both as too fine grained. The existing design already distributes whole candidate routes across up to 128 workers. A second level of parallelism would require reductions and extra barriers for every candidate, while the final reduction itself has at most 127 comparisons at 128 threads. That synchronization overhead is much larger than the small amount of work it could remove.

On GHC, W improves monotonically through eight threads; there is no measured drop-off in that range. On PSC, however, W eventually plateaus or regresses at high thread counts. We attribute that behavior to the combination of the one-wire-at-a-time work limit and repeated synchronization, with the observed cache/memory pressure making the plateau worse once each wire has too little candidate work to amortize the overhead.

3 Across-Wires: Design and Performance Debugging Journey

3.1 Implemented optimizations and final parallelization

A mode uses persistent threads instead of restarting them for every annealing iteration. In an iteration, a worker repeatedly claims one batch of wires. It first selects a replacement route for every wire in that batch, without changing occupancy. It then commits all changed wires in that batch by atomically removing each old route and atomically inserting its replacement. Different batches can be selected and committed concurrently. The worksharing barrier at the end of the iteration ensures every batch has committed before selection for the next iteration begins. Unlike W mode, a worker leaves a wire’s old route in occupancy while selecting its replacement, because another worker may be scoring that same occupancy cell. The implemented optimizations are:

- **Batch of wires per worker.** A whole batch is the unit of parallel work. Dynamic scheduling assigns the next unclaimed batch to the next available worker, which helps when wires have different candidate counts.
- **Persistent threads.** Threads are created once rather than once per annealing iteration. This avoids repeatedly creating and destroying OpenMP workers.
- **Private random state and reusable route updates.** Each worker has its own random-number generator and remembers its chosen routes

in a reusable private buffer. The random generator performs annealing’s probabilistic candidate choice; keeping it private avoids a shared-state lock and repeated allocation while scoring a batch.

- **Atomic occupancy accesses.** Workers atomically read both occupancy views while scoring and atomically decrement/increment them when committing routes. These operations prevent C++ data races while allowing separate batches to make progress concurrently.
- **Overlap intervals instead of per-cell route membership tests.** The first A scorer called `cell_on_wire()` at every candidate cell to find whether it overlapped the old route. The final scorer computes the small set of segment-overlap ranges once, then scores contiguous overlapping and non-overlapping spans directly.
- **Correct overlap scoring.** A candidate cell shared with the old route uses $value^2$, since removing the old route and adding the candidate cancel. All other cells use $(value + 1)^2$. The interval representation applies this required A-mode rule without changing the cost function.
- **Direction-specific scoring and two occupancy views.** The shared horizontal/vertical loops use the row-major and transposed matrices so that each segment is traversed contiguously in its own direction.

3.2 Synchronization and correctness

The OpenMP worksharing loop has an implicit barrier at the end of each iteration. That is the synchronization point separating one iteration’s batch commits from the next iteration’s route selection; no global lock or critical section is used around the complete scoring loop. While a worker scores every candidate cell, it uses an atomic read from the appropriate occupancy view, because another worker may be committing a different batch at the same time. After choosing every wire in its batch, the worker commits each change with atomic updates: it decrements every cell of the old route and increments every cell of the replacement route in both the row-major and transposed occupancy views. The atomics prevent data races, but they do not make the many reads of one candidate route into one globally consistent snapshot.

We determine overlaps between each candidate segment and the old wire before visiting that segment’s cells. The scorer walks non-overlapping spans using $(value + 1)^2$ and overlapping spans using $value^2$, where removing the

old wire and adding the candidate cancel. This replaces a segment scan at every cell with a small fixed-size range calculation per segment.

3.3 Optimization Decision Making and Tradeoffs

An early one-thread A-mode run was much slower than W mode on the same input. Since that was a one-thread observation, it pointed to per-candidate scoring work rather than lock contention or a poor distribution of batches. We used that observation to prioritize the following decisions.

- **Overlap intervals.** The original scorer checked whether every candidate cell lay on the old route. We replaced that repeated geometric work with a constant-size overlap calculation per candidate segment. The scoring-only diagnostic below was the direct evidence for this change.
- **Dynamic, one-wire batches.** Wires whose endpoints are farther apart have more candidate routes to consider, so a dynamic work queue avoids leaving a worker with a long wire while others are idle. We retained batch size one because larger batches defer each wire's update and did not provide a robust performance benefit in our experiments. A guided-scheduling trial was also within normal run-to-run variation, so we kept the simpler dynamic policy.
- **Persistent workers, private RNGs, and reusable update buffers.** These choices remove repeated thread startup, shared random-state locking, and per-batch allocations. They are low-risk overhead reductions that do not alter the candidate set or the A-mode update semantics.
- **Atomic accesses remain enabled.** Atomics are required while independent batches can score and commit concurrently. We tried a special single-thread path without them, but the measured cycle improvement was only about 1.5%; we reverted that extra code rather than complicating the final implementation.
- **Two directional occupancy views.** The scorer dominates A-mode runtime, so keeping horizontal and vertical scans contiguous is worthwhile even though each commit must update both views.

Scoring variant	Local median (s)
W-style scoring	0.159
W-style scoring with atomic reads	0.467
Initial A: atomic reads and per-cell membership checks	1.63
Updated A: atomic reads and segment overlap ranges	0.428

Table 1: Scoring-only diagnostic on identical initial few-input candidates. All cost checksums matched. These timings isolate the scorer and are not full-program assignment benchmark times.

3.4 Limitations against Perfect Speedup

A mode has more available parallel work than W mode, but it still cannot achieve perfect $P\times$ speedup. Wires whose endpoints are farther apart have more candidate routes to consider, which creates unequal workloads. Dynamic scheduling reduces that imbalance, but each one-wire batch still incurs scheduling overhead and the last few long wires can leave workers idle. At sufficiently high thread counts there are also simply too few independent wire batches: the few-input case has only 128 wires, so its PSC A-mode curve levels off around 32 threads.

The other limit is shared occupancy traffic. Every candidate-cell score performs an atomic read, and every accepted route performs atomic decrements and increments in both occupancy views. When concurrent routes touch nearby cells, these operations create cache-coherence and memory-bandwidth traffic; the transposed view improves scan locality but cannot remove that sharing. The final GHC eight-thread A-mode speedups of $7.35\times$, $6.88\times$, and $6.40\times$ for few, medium, and abundant inputs reflect this combination of remaining imbalance, dynamic-scheduling overhead, and atomic/data-movement costs. PSC shows that medium and abundant inputs have enough wire-level work to scale much farther, but total speedup still falls below computation speedup once serial initialization becomes noticeable.

4 Routing Output

Figures 1 and 2 show the required medium-input routing output on eight GHC workers.



Figure 1: Medium-input routing output at eight GHC workers: within-wires (left) and across-wires (right).

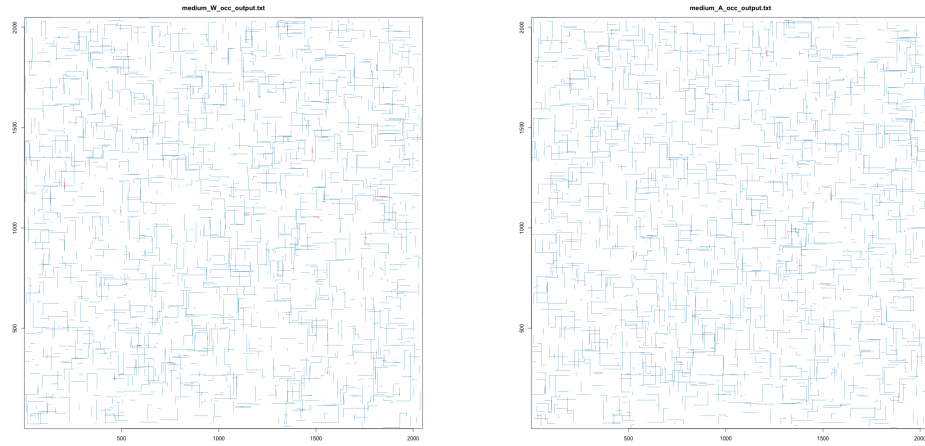


Figure 2: Graphical occupancy matrices for the same medium-input eight-worker runs: within-wires (left) and across-wires (right). White, blue, red, and purple denote occupancy 0, 1, 2, and 3, respectively.

5 Experimental Results from the GHC Machines

We collected the final GHC data on `ghc39` with `OMP_PLACES=cores` and `OMP_PROC_BIND=close`. Every configuration used $P = 0.1$, five annealing iterations, and A-mode batch size one unless a sensitivity experiment explicitly varied P . Each timing configuration was run three times and the tables use the median initialization and computation times.

5.1 Speedup graphs

Table 2 reports final computation medians. W is slightly faster than A at one thread for medium and abundant inputs because it uses ordinary occupancy reads after removing its current wire. A instead uses atomic reads so that concurrent batch commits remain data-race free. Both modes are close to ideal through four threads. At eight threads, W is strongest on few wires ($7.70\times$) and A is strongest on few wires ($7.35\times$), while the abundant inputs show lower speedup because occupancy traffic and update contention become more significant.

Mode	Input	1 thread	2 threads	4 threads	8 threads
W	Few	2.668	1.311	0.669	0.346
W	Medium	9.302	4.610	2.396	1.291
W	Abundant	24.632	12.330	6.695	4.144
A	Few	2.601	1.300	0.674	0.354
A	Medium	9.725	4.865	2.526	1.413
A	Abundant	27.401	13.723	7.340	4.285

Table 2: Final GHC median computation times (s), three trials per configuration.

Mode	Input	S_2^{comp}	S_4^{comp}	S_8^{comp}	S_8^{total}
W	Few	2.04	3.99	7.70	7.55
W	Medium	2.02	3.88	7.20	7.16
W	Abundant	2.00	3.68	5.95	5.93
A	Few	2.00	3.86	7.35	7.21
A	Medium	2.00	3.85	6.88	6.85
A	Abundant	2.00	3.73	6.40	6.38

Table 3: Final GHC speedups. Computation speedup is C_1/C_p ; total speedup uses median initialization plus computation time.

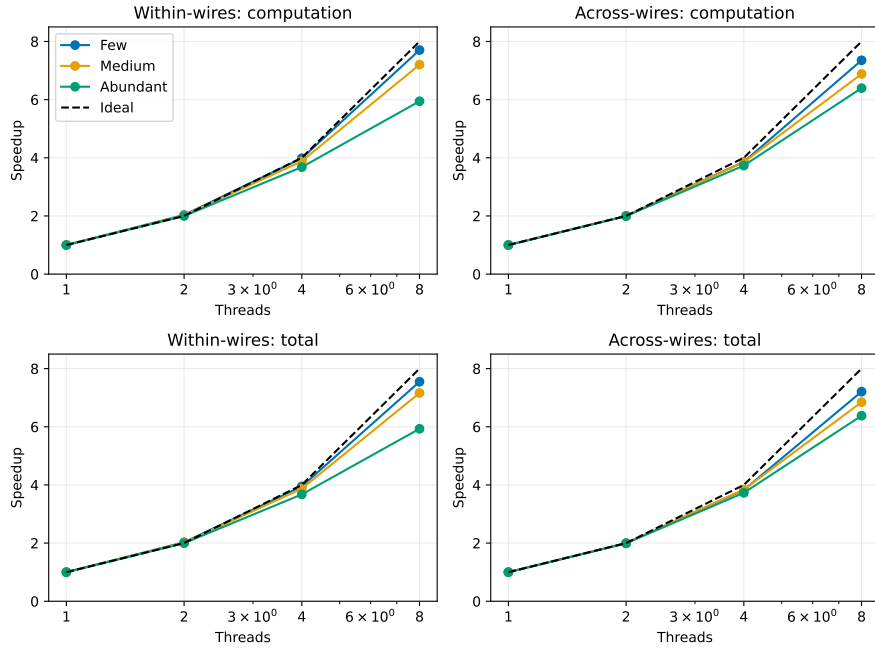


Figure 3: Required GHC speedup curves. Each curve uses the three-trial median for its input and mode; the dashed line is ideal linear scaling.

5.2 Cache-miss measurements

For each core configuration we also ran `perf stat -e cache-misses`. The counter covers the entire OpenMP process, so its reported total includes all workers. The assignment’s arithmetic mean per-worker value is therefore total misses divided by the requested worker count.

Input	Threads	W total (M)	W/worker (M)	A total (M)	A/worker (M)
Few	1	7.56	7.56	7.50	7.50
	2	7.59	3.80	7.51	3.75
	4	7.88	1.97	7.53	1.88
	8	7.70	0.96	7.75	0.97
Medium	1	27.48	27.48	28.87	28.87
	2	34.51	17.25	28.76	14.38
	4	35.45	8.86	29.25	7.31
	8	33.35	4.17	29.42	3.68
Abundant	1	114.05	114.05	120.34	120.34
	2	145.79	72.89	117.97	58.98
	4	144.97	36.24	118.92	29.73
	8	135.90	16.99	117.89	14.74

Table 4: GHC cache misses from representative `perf stat` runs. M denotes millions of misses.

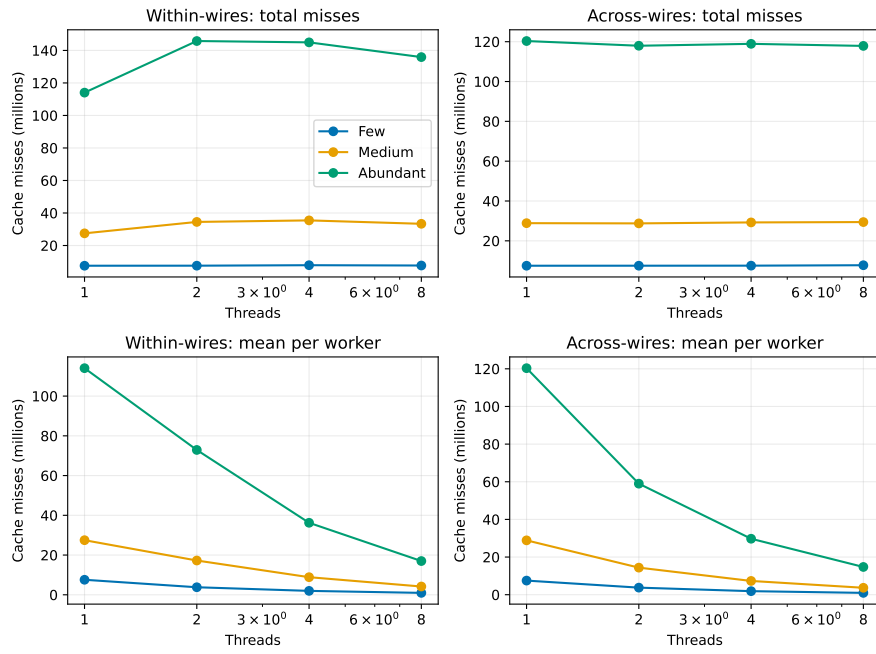


Figure 4: Required GHC cache-miss plots. Top: process-wide total misses. Bottom: arithmetic mean misses per requested worker (total divided by thread count).

The main surprise is that A-mode aggregate cache misses stay nearly flat as threads increase: on medium input they change from 28.87 million at one thread to 29.42 million at eight, and on abundant input they change from 120.34 to 117.89 million. We expected concurrent atomic reads and updates to nearby occupancy cells to create more contention and possible false sharing, which intuitively would increase the number of cache misses. In contrast, W-mode aggregate misses rise on the medium and abundant inputs, consistent with its repeated rereading of the shared occupancy arrays during separate per-wire searches.

6 Sensitivity Studies on the GHC Machines

6.1 Sensitivity to the probability of choosing a random route

The required A-mode probability sweep on medium input was stable: computation speedup at eight threads was $7.39\times$ for $P = 0.01$, $7.40\times$ for $P = 0.1$, and $7.39\times$ for $P = 0.5$. Higher P reduces absolute work (median one-thread computation times 10.697, 9.750, and 5.504 seconds), but does not materially change parallel efficiency. A larger P replaces more exhaustive candidate searches with one random-route choice. This probability is sampled independently for every wire decision by the worker that owns that wire, so all runs both replace roughly the same fraction of their expensive searches. The work decreases at both thread counts, leaving their speedup ratio nearly unchanged.

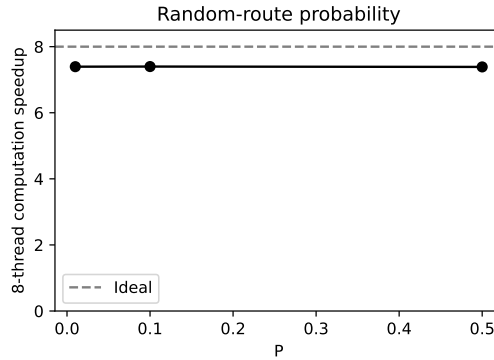


Figure 5: A-mode medium-input sensitivity to the random-route probability.

6.2 Sensitivity to problem size

For the problem-size studies, eight-thread A-mode speedups were $7.20\times$, $6.39\times$, and $7.36\times$ for grid dimensions 512, 2048, and 8192. With 256, 1024, and 4096 wires on the 2048 grid, they were $7.39\times$, $7.27\times$, and $6.38\times$, respectively. Thus A mode generally has enough independent wire work to scale well at eight workers; the largest wire-count case shows the expected loss from greater occupancy interference. The grid-size trend is not monotonic because larger grids simultaneously change the candidate work per wire and the occupancy working set. In contrast, adding more wires directly creates more concurrent atomic accesses to the same occupancy matrix, making the decline at 4096 wires consistent with increased contention.

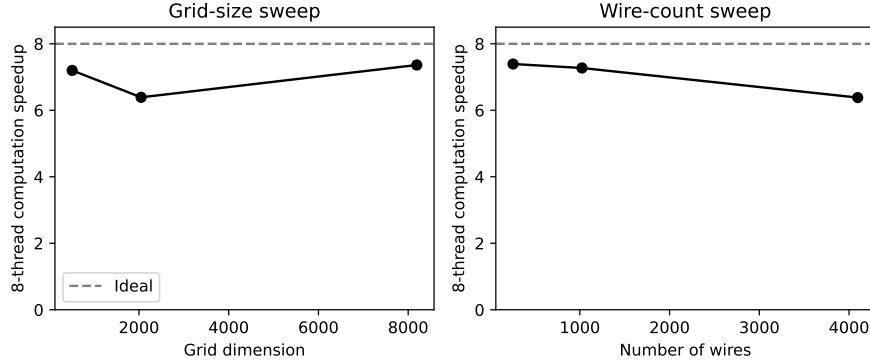


Figure 6: A-mode problem-size sensitivity. The dashed line marks ideal eight-thread speedup.

7 Experimental Results from the PSC Machines

We benchmarked on a dedicated Bridges-2 Regular-Memory node. For each of the three inputs and both routing modes, we ran three trials at 1, 2, 4, 8, 16, 32, 64, and 128 threads: $3 \times 2 \times 8 \times 3 = 144$ runs total. Every run used $P = 0.1$, five annealing iterations, and $B = 1$. We recorded the reported initialization and computation times, and calculated total time as their sum.

7.1 Speedup graphs

Table 5 lists the median computation time in seconds. It is the source table for the computation-speedup curves in Figure 7.

Threads	Few W	Few A	Medium W	Medium A	Abundant W	Abundant A
1	2.833	4.303	10.649	15.558	29.008	42.591
2	1.435	1.968	5.438	7.436	15.086	20.454
4	0.714	0.973	2.722	3.637	7.728	10.063
8	0.363	0.509	1.460	1.818	4.511	5.026
16	0.195	0.291	0.857	0.906	3.025	2.521
32	0.112	0.234	0.586	0.466	2.460	1.279
64	0.077	0.245	0.513	0.273	2.572	0.715
128	0.074	0.243	0.626	0.161	3.987	0.396

Table 5: PSC Bridges-2 RM median computation times (s), three trials per configuration. Every run passed validation.

Threads	Few W	Few A	Medium W	Medium A	Abundant W	Abundant A
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.97	2.19	1.96	2.09	1.92	2.08
4	3.97	4.42	3.91	4.28	3.75	4.23
8	7.81	8.45	7.30	8.56	6.43	8.47
16	14.55	14.80	12.43	17.17	9.59	16.89
32	25.20	18.35	18.16	33.36	11.79	33.30
64	36.64	17.59	20.77	57.08	11.28	59.59
128	38.37	17.70	17.01	96.36	7.27	107.60

Table 6: PSC computation speedup. Each column uses its own one-thread median as the baseline.

Threads	Few W	Few A	Medium W	Medium A	Abundant W	Abundant A
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.97	2.18	1.96	2.09	1.92	2.08
4	3.93	4.38	3.90	4.27	3.75	4.23
8	7.59	8.26	7.25	8.51	6.41	8.45
16	13.72	14.18	12.26	16.94	9.54	16.79
32	22.67	17.39	17.77	32.40	11.72	32.84
64	31.31	16.70	20.25	54.16	11.20	58.07
128	31.60	16.62	16.55	86.72	7.24	101.84

Table 7: PSC total speedup. Initialization is serial and becomes visible once computation is very short, especially for the few-input cases.

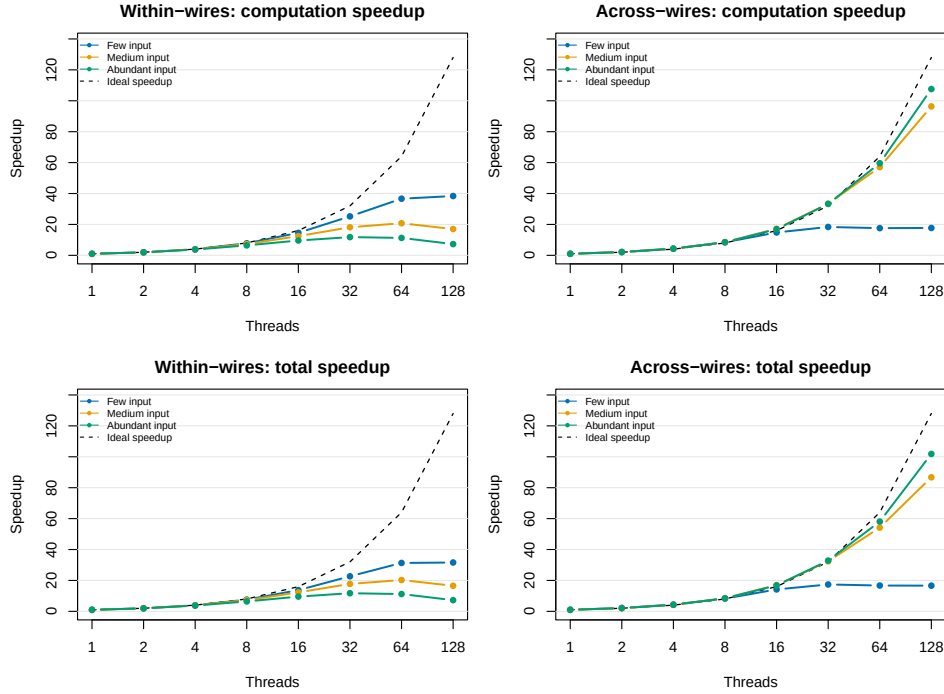


Figure 7: PSC Bridges-2 RM speedup curves derived from the median-of-three summary data. The dashed line is ideal linear speedup. W-mode plateaus when a single wire’s candidate search cannot keep all workers busy; A-mode retains a larger pool of concurrent wires on the medium and abundant inputs.

7.2 Discussion of speedup results

At eight threads, both modes scale well for every input: W obtains computation speedups of $7.81\times$, $7.30\times$, and $6.43\times$ for few, medium, and abundant inputs, while A obtains $8.45\times$, $8.56\times$, and $8.47\times$. One possible explanation for the super linear speed up is much better cache hits compared to the single thread version.

The two strategies diverge at high core counts. W peaks at 64 threads on few and medium, and at 32 threads on abundant, before flattening or regressing. For medium and abundant inputs, candidate searches for individual wires cannot always supply enough balanced work to amortize repeated parallel-region and barrier costs.

A-mode instead reaches $96.36\times$ computation speedup on medium and $107.60\times$ on abundant at 128 threads. These results are consistent with A-mode’s larger pool of concurrently selectable wires. Few-input A-mode levels off around 32 threads because only 128 wires are available and the fixed initialization cost becomes a larger fraction of total time. However, medium A-mode falls from $96.36\times$ computation speedup to $86.72\times$ total speedup at 128 threads because initialization remains largely serial and becomes a substantial fraction of the increasingly short end-to-end runtime.

Table 8 compares our PSC one-thread computation medians with the handout’s good-performance reference. W-mode is close to the reference on few and medium and is slower on abundant. A-mode’s one-thread time is higher than the reference, even though its high-core-count scaling is strong.

Input	Our W	Ref. W	Our A	Ref. A
Few	2.833	3.138	4.303	3.066
Medium	10.649	10.345	15.558	11.460
Abundant	29.008	24.817	42.591	26.707

Table 8: PSC one-thread computation time (s): our medians versus the handout’s good-performance reference.

7.3 Comparison with GHC results

At eight workers, GHC W-mode speedups are $7.70\times$, $7.20\times$, and $5.95\times$ for few, medium, and abundant inputs, compared with PSC values of $7.81\times$, $7.30\times$, and $6.43\times$. Within-wires exposes candidates of only one wire at a time, so increasingly large teams eventually encounter insufficient work, barriers, and shared-occupancy traffic.

For A mode, GHC eight-worker speedups are $7.35\times$, $6.88\times$, and $6.40\times$, whereas PSC reports $8.45\times$, $8.56\times$, and $8.47\times$. The PSC curves show the benefit of parallelizing across wires: instead of exposing only one wire's candidate routes at a time, A mode gives workers a shared pool of selectable wires. The medium and abundant inputs have enough independent wire work to continue scaling through 128 threads, unlike W mode, which plateaus earlier. The exception is the few test, with only 128 wires, it levels off around 32 threads as there aren't enough wires to saturate the rest of the threads